

Algorithm and Programming

Sorting Algorithms

Finding min number

- Assume that we have an array

```
int a[ ] = {100, 0, 95, -5, 72, 9, 1, -3, 49, 70};
```

How to find a minimum
number in this array?

Exchanging two numbers

- Assume that we have two numbers,
`int n=100, m=50;`

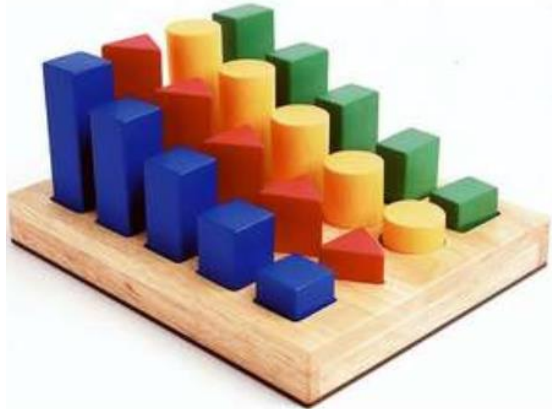
How to exchange numbers between these two variables such that we get `m=100` and `n=50` instead?

Let's create a function in C++ to do so.

What is sorting?

□ Definition

- **Sorting** is any process of arranging items systematically, with two distinct meanings:
 - **Ordering** : arranging items in a sequence ordered by some criterion
 - **Categorizing** : grouping items with similar properties



What is sorting algorithm?

□ Definition

- A **sorting algorithm** is an algorithm that puts items of a list in a certain order



How to compare elements (Examples)

- Some examples

1. Order numbers from the smallest to the largest or vice versa
2. Order words alphabetically
3. Order people alphabetically by name
 - People having the same firstname are ordered according to their lastname
 - People have the same firstname and lastname could be then ordered according to their age

- Hence, comparisons could be based on rules that are used to sort things

What do we sort?

□ Definition

- We order collections of data
- Example
 1. Array
 2. Linked list

□ Insertion sort

□ Selection sort

□ Merge sort

□ Heap sort

□ Quick sort

□ Bubble sort

□ Shell sort

□ Comb sort

□ Counting sort

□ Bucket sort

□ Radix sort

Simple sorting algorithms

Efficiency sorting algorithms

Bubble sort and variant algorithms

Distribution sort algorithms

Etc....

Common sorting algorithms

□ Remark

- There are many algorithms that we can use to order a list of elements
- Although they are all different, they share a common building block
 - ✓ Compare pairs of elements to decide which is their relative order
 - ✓ When needed, they swap the elements to restore their order

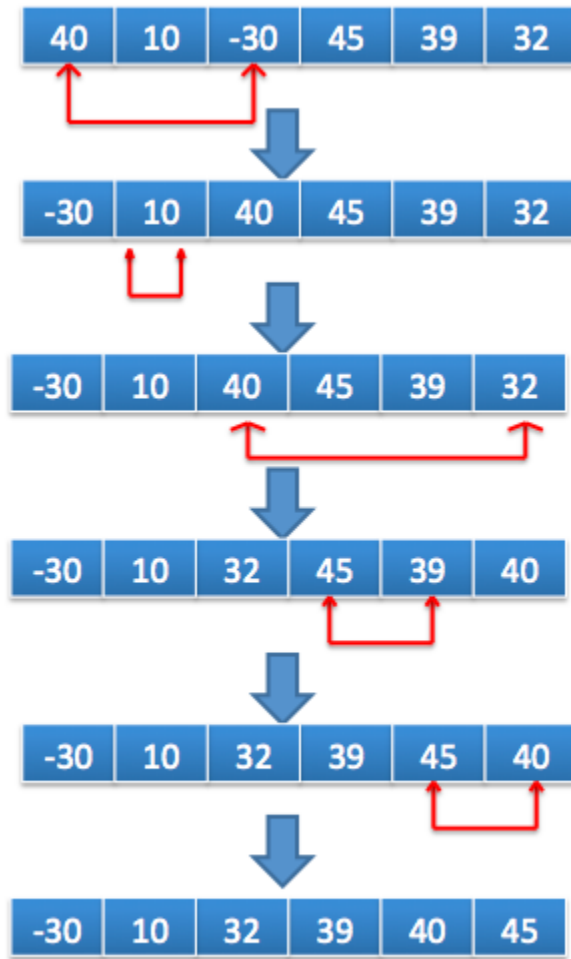
Criteria for sorting algorithms

□ Criteria

1. Total number of steps (complexity)
2. Number of comparisons performed
3. Number of interchanges performed
4. Type of memory used
 - Internal algorithms, External algorithms

The less the better!

Selection sort



Selection sort

Step 1

- Find index of min element
 - Loop to find it
- Swap element at this index with element at position i
 - Use swap function

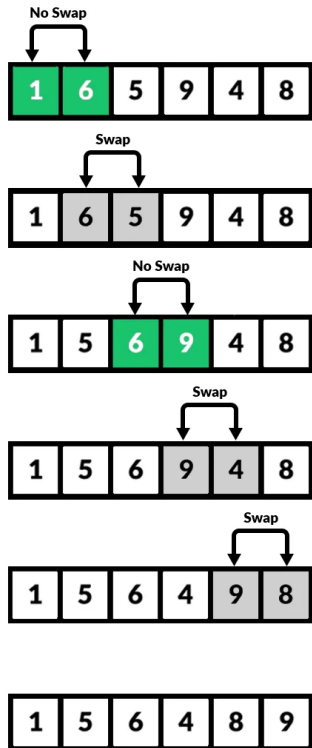
$i = i + 1$

Repeat step 1

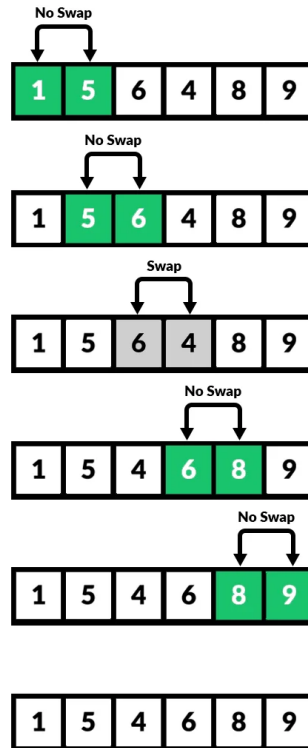
```
4 void swap(int *x, int *y){
5     int temp;
6     temp = *x;
7     *x = *y;
8     *y = temp;
9 }
10
11 void selectionSort(int arr[], int n){
12     int i, j, minIndex;
13
14     for (i = 0; i < n-1; i++){
15         // Find index of min element
16         minIndex = i;
17         for (j = i+1; j < n; j++){
18             if (arr[j] < arr[minIndex]){
19                 minIndex = j;
20             }
21         }
22
23         // Swap the min element with the element at index i
24         swap(&arr[minIndex], &arr[i]);
25     }
26 }
```

Bubble sort

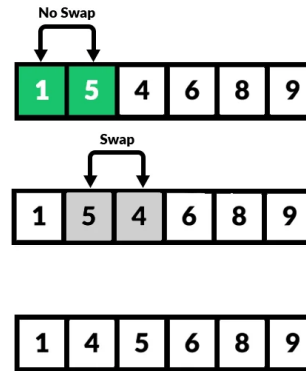
First Pass



Second Pass



Final Pass



- Step 1
 - Compare two first elements and swap if needed
 - Element at i and $i+1$
- Step 2
 - Repeat step 1
 - Element at $i+1$ and $i+2$
- If no swap, sorted already.

```
4 void swap(int *x, int *y){
5     int temp = *x;
6     *x = *y;
7     *y = temp;
8 }
9
10 void bubbleSort(int arr[], int n){
11     int i, j;
12     bool state;
13
14     for (i=0; i < n-1; i++){
15         state = false;
16         for (j=0; j < n-i-1; j++){
17             if (arr[j] > arr[j+1]){
18                 swap(&arr[j], &arr[j+1]);
19                 state = true;
20             }
21         }
22
23         // IF no swap, then break loop since all data are already sorted!!
24         if (state == false){
25             break;
26         }
27     }
28 }
```

Complexity of Algorithm (time consuming)

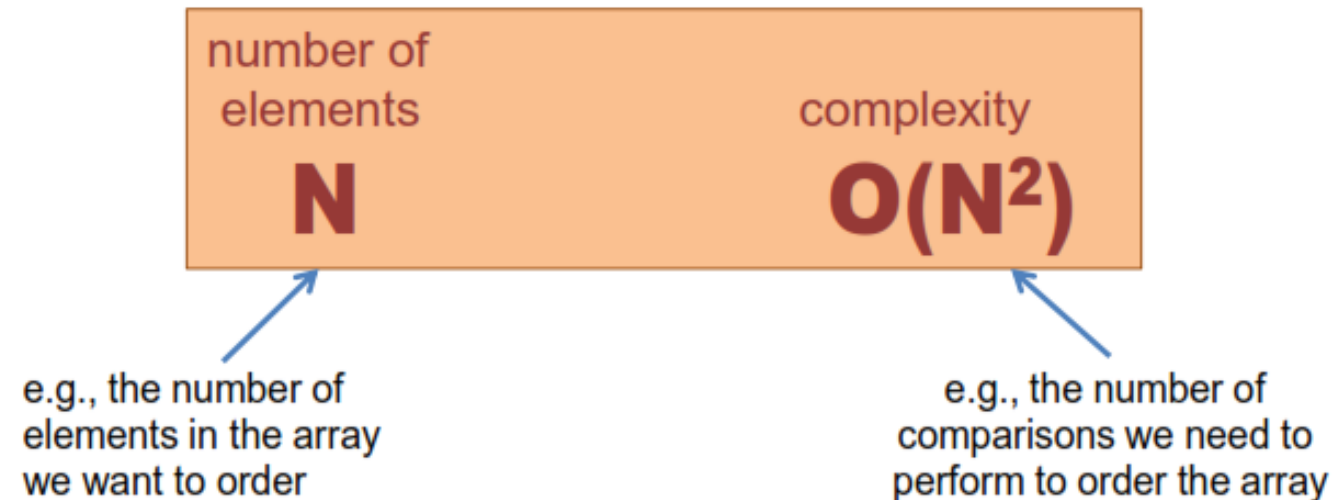
□ Definition

- **Time complexity of an algorithm** is the amount of time taken by the algorithm to run
- Time complexity
 - Is expressed as a function of the length of the input
 - Is commonly expressed using **Big-O Notation**

Complexity of Algorithm

□ Big-O Notation

- Big-O Notation describes the limiting behavior of a function when the number of elements it has to process tends to infinity
- The concept is simplified in this way:



Complexity of Algorithm

□ Big-O Notation

- Which algorithm is the best?
 - *To us, algorithm is good if it consumes less time to run*
- Let N be the number of elements we want to process
 - E.g: number of elements in an array
- What if we have 4 algorithms whose time complexities are as follows?

Algorithm	Time Complexity
Algorithm 1	$O(N)$
Algorithm 2	$O(N^2)$
Algorithm 3	$O(\log(N))$
Algorithm 4	$O(N \cdot \log(N))$

Complexity of Algorithm

□ Big-O Notation

- These time complexity can be treated as if it were a mathematical function

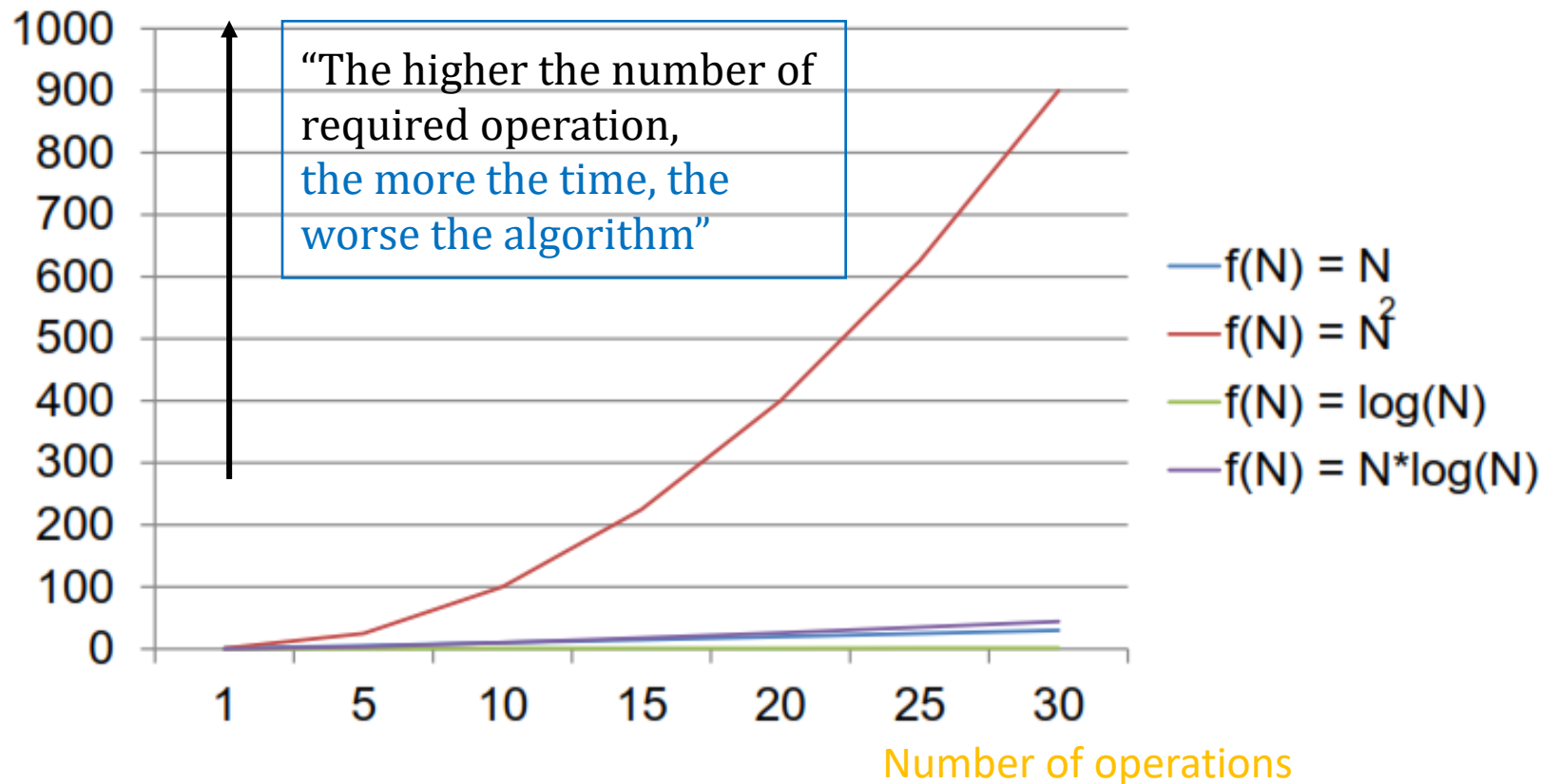
Algorithm	Time Complexity	Math function
Algorithm 1	$O(N)$	$f(N) = N$
Algorithm 2	$O(N^2)$	$f(N) = N^2$
Algorithm 3	$O(\log(N))$	$f(N) = \log(N)$
Algorithm 4	$O(N \cdot \log(N))$	$f(N) = N \cdot \log(N)$

Complexity of Algorithm

□ Big-O Notation

- These time complexity can be treated as if it were a mathematical function

Time (ms)



Algorithm	Time Complexity	Math function
Algorithm 1	$O(N)$	$f(N) = N$
Algorithm 2	$O(N^2)$	$f(N) = N^2$
Algorithm 3	$O(\log(N))$	$f(N) = \log(N)$
Algorithm 4	$O(N \cdot \log(N))$	$f(N) = N \cdot \log(N)$

Sorting Algorithm Techniques

❑ Following are some families of sorting algorithms

- *Simple sorts*
- Efficient sorts
- Bubble sort and variants
- Distribution sorts

Simple sorting algorithms are efficient on small data amounts but generally do not perform well on large lists

- ❑ Insertion sort
- ❑ Selection sort

Sorting Algorithm Techniques

❑ Following are families of sorting algorithms

- Simple sorts
- *Efficient sorts*
- Bubble sort and variants
- Distribution sorts

Efficient sorting algorithms are those algorithms whose average complexity is the best you can find $O(N \log(N))$

- ❑ Merge sort
- ❑ Heap sort
- ❑ Quick sort

Sorting Algorithm Techniques

❑ Following are families of sorting algorithms

- Simple sorts
- Efficient sorts
- *Bubble sort and variants*
- Distribution sorts

Bubble sorting algorithms is very simple, and the same characteristics is inherited by all its variants. However, it is highly inefficient (high time complexity $O(N^2)$)

- ❑ Bubble sort
- ❑ Shell sort
- ❑ Comb sort

Sorting Algorithm Techniques

❑ Following are families of sorting algorithms

- Simple sorts
- Efficient sorts
- Bubble sort and variants
- *Distribution sorts*

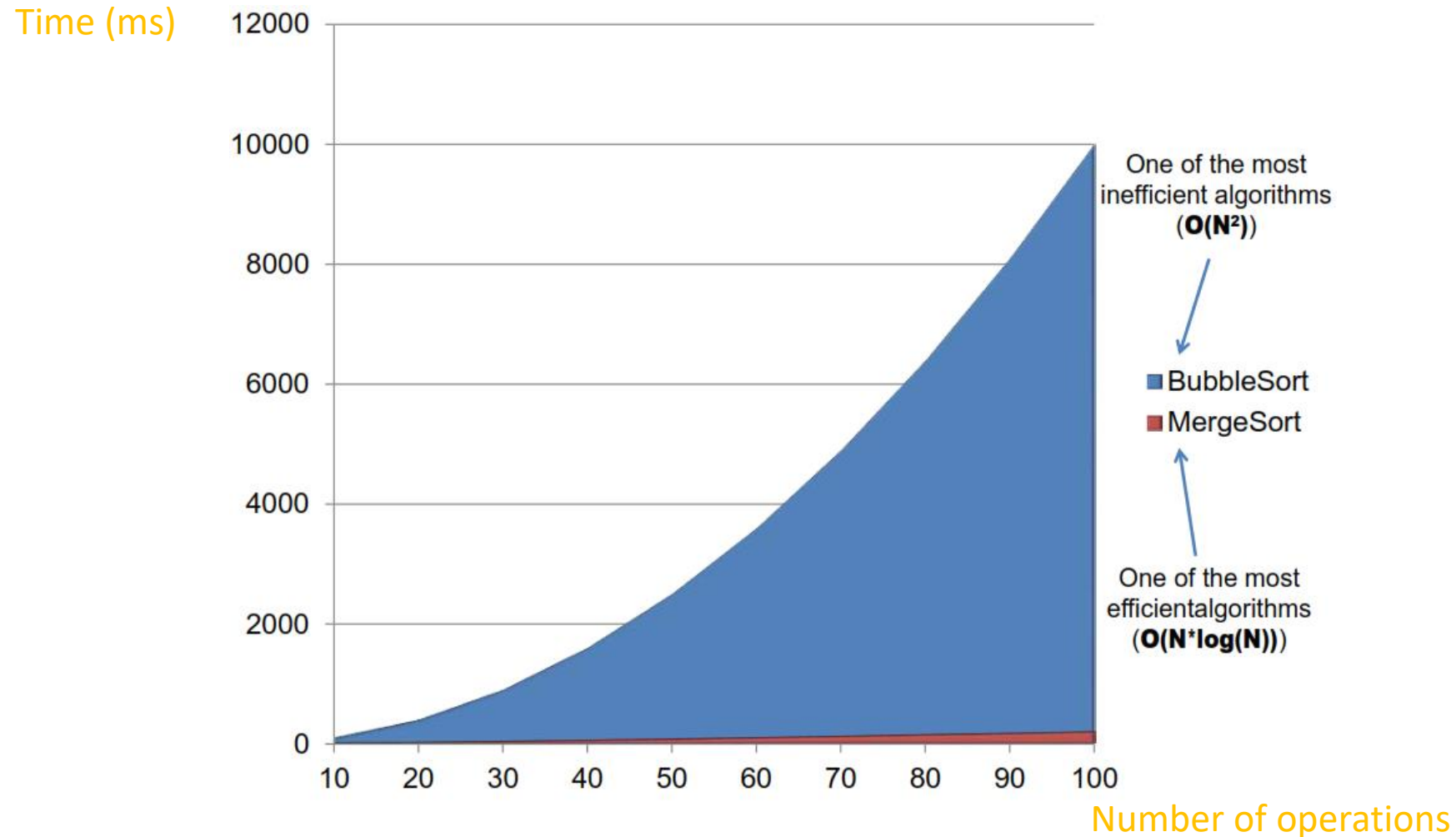
Distribution sorting algorithms distribute the input to intermediate structures, which are gathered and placed on the output. They are useful in case of very large data sets that do not fit in memory.

Intermediate structures can be deployed on different machine

- ❑ Counting sort
- ❑ Bucket sort
- ❑ Radix sort

Sorting Algorithm Techniques

□ Performance: Bubble Vs. Merge sort



Sorting Algorithm Techniques

☐ In summary

- The two best families of sorting algorithms are
 - **Efficient sorts** : have best average time complexity $O(N \log(N))$
 - ☐ Merge sort
 - ☐ Heap sort
 - ☐ Quick sort
 - **Distribution sorts** : perform wells on large data
 - ☐ Counting sort
 - ☐ Bucket sort
 - ☐ Radix sort

Sorting algorithms

Sorting Algorithm	Definition	Complexity	Stability?
Insertion sort	is a simple sorting algorithm that builds the final sorted array one item at a time . Method: Insertion	Worst case: n^2 Average case: n^2 Best case: n	Yes
Selection sort	is a sorting algorithm, specifically an in-place comparison sort . inefficient on large lists, and performs worse than the similar insertion sort. Method: Selection	Worst case: n^2 Average case: n^2 Best case: n^2	No
Merge sort	is an efficient, general-purpose, comparison-based sorting algorithm. It is a divide and conquer algorithm Method: Merging	Worst case: $n \log(n)$ Average case: $n \log(n)$ Best case: $n \log(n)$	Yes
Quick sort	is an efficient sorting algorithm, serving as a systematic method for placing the elements of an array in order . Method: Partitioning	Worst case: n^2 Average case: $n \log(n)$ Best case: $n \log(n)$	No
Bubble sort (or called Sinking sort)	is a simple sorting algorithm that repeatedly steps through the list, compares adjacent pairs and swaps them if they are in the wrong order. The pass through the list is repeated until the list is sorted. Method: Exchanging	Worst case: n^2 Average case: n^2 Best case: n	Yes

Q&A